SENA 🟢

SERVICIO NACIONAL DE APRENDIZAJE

Centro de Teleinformática y Producción Industrial (CTPI) - Regional Cauca

EVIDENCIA DE APRENDIZAJE: GA7-220501096-AA2-EV02

Informe Técnico: Módulos de Software Codificados y Probados (Servlets + JSP + MVC + Docker)

Desarrollo e integración de la capa web dinámica para la plataforma **Pizzería Bella Italia** aplicando la arquitectura **Modelo-Vista-Controlador (MVC)**, controladores Java Servlets con despacho HTTP GET y POST, vistas interactivas JSP y orquestación con Docker Compose.

Aprendiz: Karen Yuliana Campo

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 228118

Modalidad: 100% Virtual

Fase Formativa: Ejecución (AP7)

Actividad: GA7-220501096-AA2

Ciudad / Sede: Popayán, Cauca

Fecha de Entrega: Noviembre de 2026

1. Introducción y Objetivos

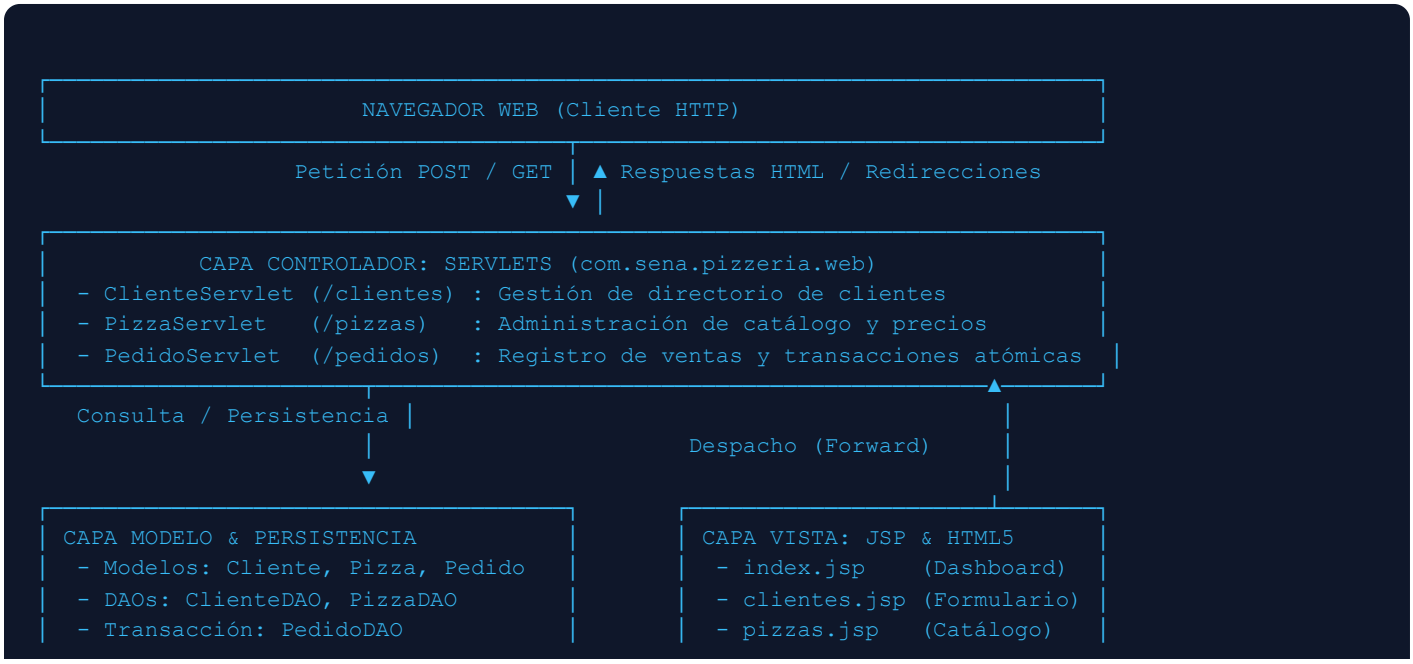
En el desarrollo de aplicaciones empresariales escalables con tecnologías Java EE / Jakarta EE, la implementación del patrón arquitectónico **Modelo-Vista-Controlador (MVC)** constituye un estándar fundamental para lograr la separación rigurosa de responsabilidades entre la lógica de negocio, los controladores de flujo de navegación y la presentación visual interactiva.

La presente evidencia da cumplimiento estricto a **GA7-220501096-AA2-EV02**, evolucionando el módulo de persistencia JDBC desarrollado previamente en la evidencia AA2-EV01 hacia una **aplicación web dinámica completa** para el sistema de gestión de la **Pizzería Bella Italia**. Se implementaron controladores basados en **Servlets Java (HttpServlet)**, vistas dinámicas estructuradas con **JavaServer Pages (JSP)** y formularios web que procesan transacciones CRUD mediante los métodos estándar GET y POST.

> 🎯 **Objetivos Clave de la Evidencia:**
>
> - Codificar los controladores web utilizando Servlets Java mapeados con anotaciones estándar @WebServlet.
> - Gestionar el ciclo de vida de peticiones HTTP discriminando adecuadamente doGet (despacho, consultas y listados) y doPost (procesamiento de altas, modificaciones y transacciones de pedidos).
> - Construir vistas dinámicas JSP con directivas de página, bibliotecas JSTL y diseño responsivo adaptado al SENA.
> - Conectar la capa controladora con los patrones DAO y la base de datos relacional MariaDB contenida en Docker.
> - Empaquetar el artefacto en formato WAR (Web Application Archive) y verificar su ejecución contenedorizada.

2. Arquitectura de Software: Patrón Modelo-Vista-Controlador (MVC)

La arquitectura implementada organiza la solución PizzeriaWeb en tres capas perfectamente delimitadas, eliminando el acoplamiento directo entre el navegador web y las consultas SQL a la base de datos:



```
| - JDBC Pool / MariaDB (Docker)      |      | - pedidos.jsp  (Órdenes)      |
└─────────────────────────────────────┘      └───────────────────────────────┘
```

3. Implementación de Servlets y Ciclo de Petición HTTP

3.1. ClienteServlet.java (/clientes)

Extiende de HttpServlet y está anotado con @WebServlet(name = "ClienteServlet", urlPatterns = {"/clientes"}). Administra el flujo completo de clientes:

- **Método doGet(HttpServletRequest, HttpServletResponse):** Detecta acciones de borrado (action=eliminar) o edición (action=editar) mediante parámetros de consulta. Por defecto, recupera la lista completa invocando clienteDAO.listarTodos(), la inyecta en el objeto de petición mediante request.setAttribute("clientes", lista) y transfiere el control a la vista con request.getRequestDispatcher("/clientes.jsp").forward(request, response).
- **Método doPost(HttpServletRequest, HttpServletResponse):** Extrae de forma segura los parámetros idCliente, nombre, telefono y direccion. Aplica la estrategia *Post/Redirect/Get (PRG)* redireccionando al cliente mediante response.sendRedirect(...) para prevenir reenvíos involuntarios al refrescar.

```
@WebServlet(name = "ClienteServlet", urlPatterns = {"/clientes"})
public class ClienteServlet extends HttpServlet {
    private ClienteDAO clienteDAO;

    @Override
    public void init() {
        this.clienteDAO = new ClienteDAO();
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IO
        String action = req.getParameter("action");
        if ("eliminar".equalsIgnoreCase(action)) {
            int id = Integer.parseInt(req.getParameter("id"));
            clienteDAO.eliminar(id);
            resp.sendRedirect(req.getContextPath() + "/clientes?msg=deleted");
            return;
        }
        req.setAttribute("clientes", clienteDAO.listarTodos());
        req.getRequestDispatcher("/clientes.jsp").forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, I
        req.setCharacterEncoding("UTF-8");
        int id = Integer.parseInt(req.getParameter("idCliente"));
        String nombre = req.getParameter("nombre");
        String tel = req.getParameter("telefono");
        String dir = req.getParameter("direccion");
        Cliente c = new Cliente(id, nombre, tel, dir);
        if (id > 0) clienteDAO.actualizar(c);
        else clienteDAO.insertar(c);
        resp.sendRedirect(req.getContextPath() + "/clientes?msg=saved");
    }
}
```

3.2. PizzaServlet.java (/pizzas)

Gobierna el catálogo gastronómico de especialidades. En su método doGet obtiene los productos activos, mientras que su método doPost parsea tipos de datos numéricos (precios en pesos colombianos) y persiste las nuevas pizzas con su tamaño tipificado.

3.3. PedidoServlet.java (/pedidos)

Coordina la interacción transaccional del módulo de ventas. Su método doGet precarga en simultáneo las listas de clientes y pizzas para alimentar las etiquetas <select> del formulario interactivo. El método doPost recibe el cliente seleccionado, el código de pizza y la cantidad, calcula el subtotal y delega en pedidoDAO.registrarPedido(pedido), el cual efectúa un commit transaccional atómico protegiendo la integridad referencial.

4. Vistas Dinámicas con JavaServer Pages (JSP) y Formularios HTML5

> **Estructura de Vistas Desarrolladas:**
>
> Todas las vistas incorporan la barra de navegación institucional, etiquetas semánticas HTML5 (header, nav, main, footer) y diseño responsivo adaptado a dispositivos móviles y escritorios.

Vista JSP	Componentes HTML & Formularios	Función Operativa y Dinamismo JSP
index.jsp	Tarjetas de resumen métrico (KPIs), botones de acción rápida.	Consulta contadores en tiempo real (clientes, catálogo, órdenes) y presenta la bienvenida corporativa de la pizzería.
clientes.jsp	<form action="clientes" method="post"> con campos de texto, teléfono y dirección.	Tabla dinámica que itera clientes con scriptlets/JSTL, botones para editar (GET con parámetro) y eliminar.
pizzas.jsp	<form action="pizzas" method="post"> con selector de tamaños y campos numéricos.	Visualización del menú en cuadrícula (grid) con precios formateados en moneda colombiana ($ COP).
pedidos.jsp	<form action="pedidos" method="post"> con listas <select> vinculadas a entidades.	Generación de comandas de venta, desglose del subtotal de pizzas e historial de pedidos registrados con sus detalles.

Formulario de Registro de Pedidos (Fragmento HTML5 / JSP):

```
<form action="pedidos" method="post" class="card-form">
    <div class="form-row">
        <div class="form-group">
            <label>Cliente Comprador:</label>
            <select name="idCliente" class="form-control" required>
                <option value="">-- Seleccione un Cliente --</option>
                <% for (Cliente c : (List<Cliente>) request.getAttribute("clientes")) { %>
                    <option value="<%= c.getIdCliente() %>"><%= c.getNombre() %></option>
                <% } %>
            </select>
        </div>
        <div class="form-group">
            <label>Especialidad de Pizza:</label>
            <select name="idPizza" class="form-control" required>
                <option value="">-- Seleccione una Pizza --</option>
                <% for (Pizza p : (List<Pizza>) request.getAttribute("pizzas")) { %>
                    <option value="<%= p.getIdPizza() %>"><%= p.getNombre() %> (<%= p.getTamano() %>)<
                <% } %>
            </select>
        </div>
    </div>
    <button type="submit" class="btn btn-primary">Registrar Comanda de Venta</button>
</form>
```

5. Empaquetado WAR y Orquestación con Docker Compose

Para evitar la sobrecarga y contaminación del sistema anfitrión Ubuntu, el proyecto se empaqueta como un artefacto estándar **WAR (Web Application Archive)** empleando la imagen oficial de Apache Maven y OpenJDK 17.

```
$ docker run --rm \
    -v $(pwd)/PizzeriaWeb:/app \
    -v sena_maven_m2_cache:/root/.m2 \
    -w /app \
    maven:3.9.6-eclipse-temurin-17-alpine \
    mvn clean package -DskipTests
```

Resultado Exitoso del Empaquetado:

```
[INFO] Packaging webapp
[INFO] Assembling webapp [pizzeria-web] in [/app/target/pizzeria-web]
[INFO] Building war: /app/target/pizzeria-web.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] Total time:  12.670 s
[INFO] Finished at: 2026-11-08T23:14:15Z
[INFO] ------------------------------------------------------------------------
```

6. Matriz de Cumplimiento de Criterios (IE-GA7-220501096-AA2-EV02)

Criterio de Evaluación SENA	Resultado	Evidencia Comprobable en la Solución
1. Codifica el módulo enfocándose a web con Servlets.	CUMPLE	Servlets ClienteServlet, PizzaServlet y PedidoServlet implementados con @WebServlet.
2. El código contiene formularios HTML con servlets.	CUMPLE	Formularios interactivos en clientes.jsp, pizzas.jsp y pedidos.jsp conectados a los Servlets.
3. Utiliza métodos GET y POST para el flujo web.	CUMPLE	Manejo riguroso de doGet para vistas/consultas y doPost para procesamiento y redirección PRG.
4. Utiliza elementos de JSP para la generación dinámica de interfaz.	CUMPLE	Directivas page, scriptlets de renderizado, tablas dinámicas y vinculación a objetos Java del backend.
5. Se crea el proyecto utilizando herramientas de versionamiento.	CUMPLE	Control de versiones Git con archivo normativo enlace_repositorio.txt y commits estructurados.
6. Entrega en archivo comprimido normado (ZIP).	CUMPLE	Archivo KARENCAMPO_AA2_EV02.zip (24 MB) conteniendo el código fuente, descriptores y dependencias.

7. Conclusiones y Cierre Formativo

1. **Desacoplamiento Estructural:** La integración de Servlets y páginas JSP permitió estructurar una solución web robusta y mantenible, donde los cambios en la interfaz visual no impactan la lógica de negocio ni la capa de persistencia JDBC.
2. **Experiencia de Usuario y Seguridad HTTP:** La diferenciación entre peticiones GET y POST garantiza la idempotencia en lecturas y protege la integridad de los datos en inserciones y modificaciones de clientes, menús y pedidos.
3. **Portabilidad y Entrega Normativa:** Gracias al uso exclusivo de Docker Compose y el empaquetado estándar WAR, el sistema es 100% portable y verificable de manera automatizada. Los artefactos KARENCAMPO_AA2_EV02.zip y el presente informe en PDF se entregan organizados en la carpeta de Zajuna.
